

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในโครงงาน

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการวิจัย และพื้นฐานของระบบการจัดการโรงพยาบาลด้วยอุปกรณ์ประมวลผลแบบพกพา ซึ่งเนื้อหาในบทนี้จะกล่าวถึงทฤษฎีที่เกี่ยวข้องกับการพัฒนาระบบด้วย .NET Framework ทฤษฎีที่เกี่ยวข้องกับ Web Application และอื่นๆ ที่เกี่ยวข้องกับระบบ ซึ่งเนื้อหาทั้งหมดในบทนี้จำเป็นสำหรับการศึกษา เพื่อที่จะนำไปพัฒนาระบบต่อไปได้

2.1 สถาปัตยกรรม .NET Framework

2.1.1 .Net Framework คืออะไร

.Net Framework คือ โครงสร้างการพัฒนาโปรแกรมคอมพิวเตอร์ ที่ถูกออกแบบมาเพื่อให้อำนวยความสะดวกในการพัฒนาโปรแกรมสมัยใหม่ ที่ใช้งานในระบบเครือข่าย (Internet, Intranet, Mobile Devices, ฯลฯ) Bill Gates และ Steve Ballmer ได้บรรยายสรุปวิสัยทัศน์ ที่เกี่ยวกับ .Net เอาไว้ 3 ข้อหลัก ๆ ได้แก่

1. การพัฒนาโปรแกรมในรูปแบบของ Web Service จะเป็นหัวใจหลักในการขับเคลื่อนโปรแกรมต่าง ๆ ที่ใช้งานบน Internet. Web Service จะช่วยให้การติดต่อสื่อสารระหว่าง application บน Internet นั้นง่ายขึ้น และเป็นระบบมากยิ่งขึ้น
2. Web Service ขึ้นพื้นฐานเช่น การตรวจสอบ user ที่ log in เข้าสู่ระบบ จะถูกพัฒนาให้เป็นมาตรฐาน และสามารถนำไปใช้ได้ทั่วไปบน Internet
3. PC (desktop, notebook) และ Mobile Device ที่ต่อเชื่อมกับ Internet ได้ เช่น PDA และ โทรศัพท์มือถือ จะมีบทบาท และประโยชน์มากขึ้นไปอีก เมื่อสามารถติดต่อใช้งานโปรแกรมต่าง ๆ บน Internet ได้

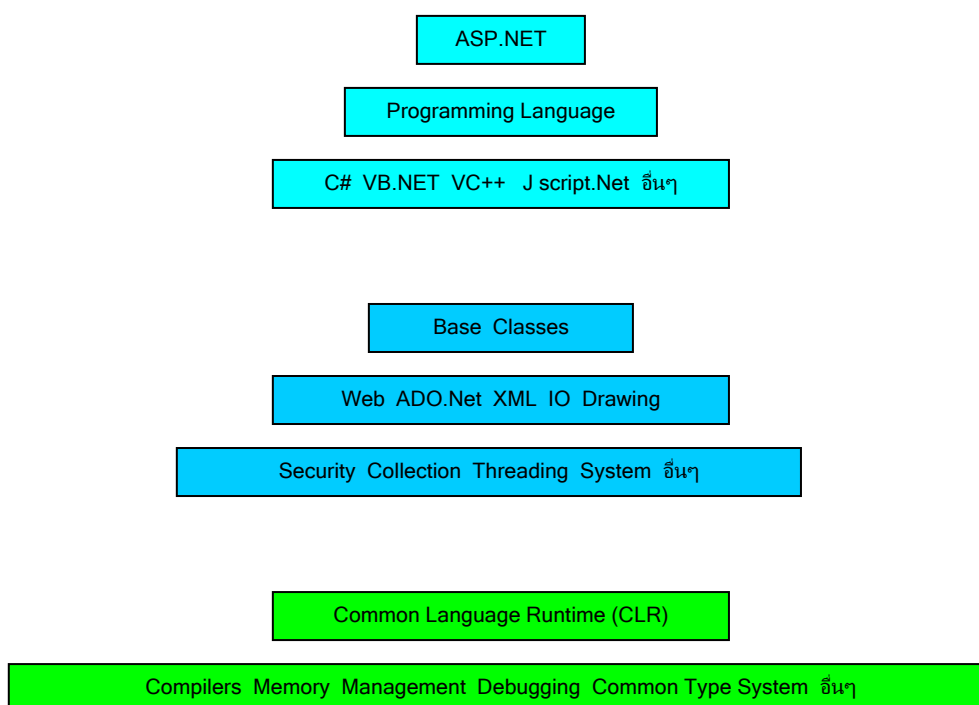
ยกตัวอย่างง่าย ๆ ว่าหากเรามีระบบตรวจสอบลายนิ้วมือ แล้วเราเอานิ้วไปประทับไปที่หน้าจอในรถแท็กซี่ Web Service ก็เกิดขึ้นในทันที เริ่มจาก ระบบตรวจสอบลายนิ้วมือซึ่งให้บริการตรวจสอบว่า ลายนิ้วมือของผู้โดยสารนั้นเป็นใคร (User-Authentication Web Service) พอทราบแล้วว่าเป็นใคร ระบบก็จะส่งข้อมูลไปยังบริษัทที่ให้บริการรถแท็กซี่ ซึ่งจะทำให้การคิดคำนวณค่าบริการ และส่ง request ไปยังธนาคารที่ผู้โดยสารมีบัญชีอยู่ เพื่อทำการหักค่าใช้จ่ายจากบัญชีของผู้โดยสาร ไปเข้าบัญชีของบริษัทรถแท็กซี่ ส่วนที่พนักงานขับรถจะได้จากการบริการ ก็จะถูกบันทึกไว้ในระบบข้อมูลพนักงานคนนั้น ๆ จะเห็นได้ว่าจากสถานการณ์ที่ได้ยกตัวอย่างไปนี้ จำเป็นที่จะต้องใช้อุปกรณ์ และการประมวลผลจากเครื่องคอมพิวเตอร์ และโปรแกรมต่าง ๆ อยู่หลายที่ ซึ่งแต่ละโปรแกรมนั้นก็อาจจะทำงานอยู่บนระบบที่แตกต่างกันไป

เช่น Windows, Linux, Mainframe, ฯลฯ ภาษาคอมพิวเตอร์ที่ใช้พัฒนาโปรแกรมเหล่านั้นก็อาจจะแตกต่างกันออกไป ระบบฐานข้อมูลก็ไม่เหมือนกัน การที่จะทำให้ระบบหลาย ๆ ระบบทำงานต่อเชื่อมกันได้อย่างราบรื่นนั้นไม่ง่ายเลย

Microsoft จึงได้พัฒนารูปแบบการพัฒนากระบวนคอมพิวเตอร์ขึ้นมา ซึ่งเรียกว่า .Net Framework นั่นเอง (อันที่จริงแล้ว Microsoft ไม่ได้เป็นผู้คิดค้นเรื่องพวกนี้ขึ้นมาแต่เพียงผู้เดียว ซึ่งสิ่งต่างๆ เหล่านี้มีผู้คิดค้นพัฒนาจากหลายบริษัท หลายหน่วยงาน ยกตัวอย่างเช่น Sun Microsystems, IBM, ฯลฯ หากแต่ว่า Microsoft นำแนวคิดเหล่านั้นมาออกแบบให้อยู่ในรูปแบบที่สามารถต่อเชื่อมกันได้ง่ายขึ้น เป็นระบบมากขึ้น) เพราะฉะนั้น ถ้าจะให้นิยามคำว่า .Net คงไม่สามารถชี้เฉพาะได้ว่า .Net คืออะไร เพราะจริง ๆ แล้ว .Net ประกอบไปด้วยส่วนประกอบต่าง ๆ หลายส่วนด้วยกัน ส่วนประกอบเหล่านี้ถูกออกแบบมาเพื่อให้ทำงานได้เข้ากันได้ดีขึ้น

2.1.2 .Net Framework นั้นประกอบไปด้วยอะไรบ้าง

ส่วนประกอบหลัก ๆ ของ .Net Framework แบ่งเป็นชั้น ๆ ได้ดัง diagram ต่อไปนี้



รูปที่ 2.1 แสดงส่วนประกอบของ .Net Framework

1. Common Language Runtime (CLR) เป็นส่วนพื้นฐานที่ติดต่อกับระบบปฏิบัติการ Windows ทำหน้าที่เป็น run-time environment ให้กับโปรแกรมที่เขียนขึ้นสำหรับใช้บน .Net CLR มีส่วนของ compiler ทั้งที่เป็นแบบปกติ (compile ก่อนที่จะนำโปรแกรมไปใช้) และแบบ Just-In-Time (compile เมื่อจะใช้โปรแกรมนั้น ๆ) มีส่วนของ Memory Management ที่เอาไว้สำหรับจัดสรรหน่วยความจำของเครื่องให้กับโปรแกรม รวมไปถึงการทำ Garbage Collection (การเรียกคืนหน่วยความจำที่ไม่ได้ใช้อีกต่อไป) ส่วนของ Common Type Systems (CTS) ทำให้ภาษาต่าง ๆ ที่เขียนขึ้นบน .Net สามารถทำงานร่วมกันได้ เพราะขนาด และรูปแบบของข้อมูลที่เก็บไว้นั้นเป็นรูปแบบเดียวกัน

2. Base Classes เป็น class library พื้นฐาน ที่โปรแกรมต่าง ๆ ไม่ว่าจะเขียนด้วยภาษาใดบน .Net ก็สามารถใช้ร่วมกันได้ เช่น การติดต่อระบบฐานข้อมูล (ADO.Net), การติดต่อกับ file system ของ server (IO), ฯลฯ

3. Programming Languages เป็นเซตของ ภาษาคอมพิวเตอร์ ที่ถูกออกแบบมาเพื่อการเขียนโปรแกรมบน .Net Framework ไม่ใคร่ขอพ่นนั้นเน้นไปที่ 3-4 ภาษาหลัก ๆ ได้แก่ VB.Net ซึ่งเป็นตัวที่พัฒนาต่อมาจาก VB, C# ซึ่งเป็นภาษาใหม่ที่มี syntax ใกล้เคียงกับ Java และ C++, Visual C++, และ JScript.Net ส่วนภาษาอื่น ๆ นั้น มีบริษัท หรือหน่วยงานอื่น ๆ เป็นผู้พัฒนาขึ้น ซึ่งคาดว่าจะมีเป็นสิบ ๆ ภาษา สำหรับ .Net Framework นั้นไม่ว่าจะเขียนโปรแกรมด้วยภาษาใดก็ตาม Compiler ใน CLR ก็จะ compile โปรแกรมนั้นให้อยู่ในรูปของ Intermediate Language (IL) ซึ่งจะถูกนำไปแปลเป็นภาษาเครื่อง (Native Code) อีกทีเมื่อตอนที่นำไปใช้

4. ASP.Net เป็นภาษา script ที่พัฒนาต่อมาจาก ASP ตัวเก่า เพื่อให้โปรแกรมเมอร์สามารถพัฒนา web application ให้อยู่บน .Net ได้สะดวกขึ้น ASP.Net นี้ถึงแม้จะอ้างอิงมาจาก ASP ตัวเก่า แต่ก็มี syntax หลายส่วนที่เปลี่ยนแปลงไป อย่างไรก็ตาม ผู้ที่เคยเขียน ASP มาก่อนไม่น่าจะมีปัญหาในการอ่าน และเขียน ASP มากนัก

หมายเหตุ : สำหรับผู้อ่านที่คุ้นเคยกับ Java ของ Sun มาก่อน จะเห็นได้ว่า .Net Framework กับ Sun's J2EE นั้นคล้ายกันมากทีเดียว สรุปโดยคร่าว ๆ ได้ดังนี้ CLR = JVM (Java Virtual Machine), IL = Java Bytecode, .Net base classes = Java Class Library, .Net Programming Languages = Java Language, ASP.Net = Java Server Page (JSP). ข้อแตกต่างในด้าน Architecture หลัก ๆ ก็คือ IL ของ .Net นั้นต้อง run บน Windows เท่านั้น (ไม่ใคร่ขอพ่นบอกว่า run บน OS ใหนก็ได้)

แต่อย่างที่เราทราบกันดี คงยากที่จะได้เห็นในอนาคตอันใกล้ หรือไม่ก็คงไม่ดีเท่ากับ run บน Windows) แต่ Java Bytecode นั้นสามารถ run บน OS ใดก็ได้ที่มี JVM. ส่วนในด้าน performance นั้น ไมโครซอฟท์ ได้ทำการเปรียบเทียบโดยพัฒนาโปรแกรม Pet Shop ด้วย .Net โปรแกรม Pet Shop นี้เป็น reference application ที่พัฒนาบน J2EE โดย Sun ไมโครซอฟท์แสดงให้เห็นว่า .Net นั้นทำให้ code สั้นลงหลายเท่าตัว ทำให้โปรแกรมทำงานเร็วขึ้นหลายเท่า เป็นต้น ในส่วนนี้ก็ต้องรอดูกันต่อไปว่า Sun จะพัฒนา Java อย่างไร เพื่อแก้เกมสักในจุดนี้

2.1.3 .Net Framework มีดีตรงไหน

- 1) ทำให้พัฒนาโปรแกรมได้เร็วขึ้น - มีโปรแกรมพื้นฐานส่วนมากไว้ให้ใช้เรียบร้อยแล้ว (base classes) โปรแกรมที่ถูกพัฒนาขึ้นใหม่สามารถนำไป compile เพื่อให้โปรแกรมอื่นๆ ได้ใช้อีก (reusable)
- 2) โปรแกรม reliable ขึ้น - เนื่องจากการเขียนโปรแกรมบางรูปแบบ บน .Net นั้นไม่สามารถกระทำได้ด้วยภาษาที่มีให้บน .Net ยกตัวอย่างเช่น การใช้ Pointer ในภาษา C ทำให้ลดโอกาสที่โปรแกรมจะทำอะไรผิดพลาดจนทำให้ระบบไม่สามารถทำงานต่อไปได้
- 3) ปลอดภัยมากขึ้น - เพราะว่า .Net Framework ควบคุมโปรแกรมว่า อะไรทำได้ อะไรทำไม่ได้
- 4) การนำโปรแกรมที่พัฒนาขึ้นไปใช้บน server จริงง่ายขึ้น - .Net Framework อนุญาตให้โปรแกรมที่พัฒนาขึ้นนั้นระบุข้อมูลต่าง ๆ ไว้กับ code เลย ทำให้ไม่ต้องนำไป register เหมือนที่ component ต้องทำ

2.1.4 คำศัพท์อื่น ๆ ที่น่าสนใจ

- **Web Forms** เป็นส่วนที่ใช้สร้าง interface ต่าง ๆ บน web browser ของผู้เข้าชมเว็บไซต์ web form จะทำการตรวจสอบ browser ของผู้ใช้ และสร้าง HTML ที่ถูกต้องสำหรับ browser นั้น ๆ
- **Server Controls** เป็นส่วนต่าง ๆ ที่ โปรแกรมเมอร์สามารถนำมาประกอบกันเพื่อใช้งานบนเว็บไซต์ได้ server control นั้นมีหลายประเภทได้แก่
 - **HTML Server Controls** ได้แก่ HTML Tag ทั่ว ๆ ไป เช่น input, table, ฯลฯ
 - **Web Server Controls** เป็น control ชนิดใหม่ที่ .Net มีให้ใช้ เช่น CheckBoxList, RadioButtonList, DataGrid, Calendar, ฯลฯ ข้อดีของ Server Control ก็คือ มันสามารถเก็บค่าต่าง ๆ บน form ไว้ได้โดยอัตโนมัติ ซึ่งทำให้โปรแกรมเมอร์ไม่ต้องเสียเวลาสร้าง hidden field หรือส่งค่าผ่านทาง URL.

- **Validator Controls** เป็น control ชนิดใหม่อีกเช่นกัน มีไว้ใช้สำหรับการตรวจสอบข้อมูลที่ผู้ใช้กรอกลงในฟอร์ม ว่าถูกต้องหรือไม่ก่อนที่จะถูกส่งไปประมวลผลที่ server ยกตัวอย่างเช่น การตรวจสอบว่า e-mail address ที่ผู้ใช้กรอกมานั้นอยู่ในรูปแบบที่ถูกต้องหรือไม่
(username@somedomain.com)

Server Control มีส่วนอย่างมากในการทำให้เขียนโปรแกรมได้ง่าย และเร็วขึ้น

- **Compiled Code และ Code Behind** - Compiled code นั้นเทียบได้กับ

Component ใน ASP แบบเก่า ใน .Net Framework นั้น compiled code มีชื่อเรียกว่า **Assembly** (.dll file) ซึ่งเราสามารถนำไปไว้ใน directory ที่ชื่อ bin .Net จะตรวจเช็คโดยอัตโนมัติ โดยที่เราไม่ต้องทำการ register assembly เหล่านั้น เหมือนอย่างที่เคยทำกับ component. Code behind คือการแยกส่วนของเนื้อหา และส่วนของโปรแกรม ในหน้าเว็บออกจากกันเป็นคนละไฟล์ ส่วนนี้ทำให้การเขียนโปรแกรม และแก้ไขโปรแกรมทำได้ง่ายมากขึ้นไปอีก ต่างจาก ASP ในแบบเก่า ที่ทุกอย่างผสมปนเปกันไปในหน้าเดียวกัน สำหรับรูปแบบใหม่นี้ web page จะมีนามสกุลเป็น .aspx ส่วน code behind จะมีนามสกุลเป็น .aspx.cs (ถ้าใช้ C#) หรือ .aspx.vb (ถ้าใช้ VB.Net)

สถาปัตยกรรม .NET Framework เป็นกรอบการทำงานของการเขียนโปรแกรมที่ไม่โครซอฟท์คิดขึ้นมาเพื่อรองรับการติดต่อสื่อสาร เพื่อแลกเปลี่ยนข้อมูล (Exchange Data) ระหว่างกัน หรือแลกเปลี่ยนข้อมูลระหว่างแพลตฟอร์ม (Platform) ให้มีความสมบูรณ์ยิ่งขึ้น โดยอาศัยภาษา XML (Extensible Markup Language) ทำหน้าที่เป็นตัวกลางในการแลกเปลี่ยนข้อมูลระหว่างแพลตฟอร์มไฟล์ของข้อมูล

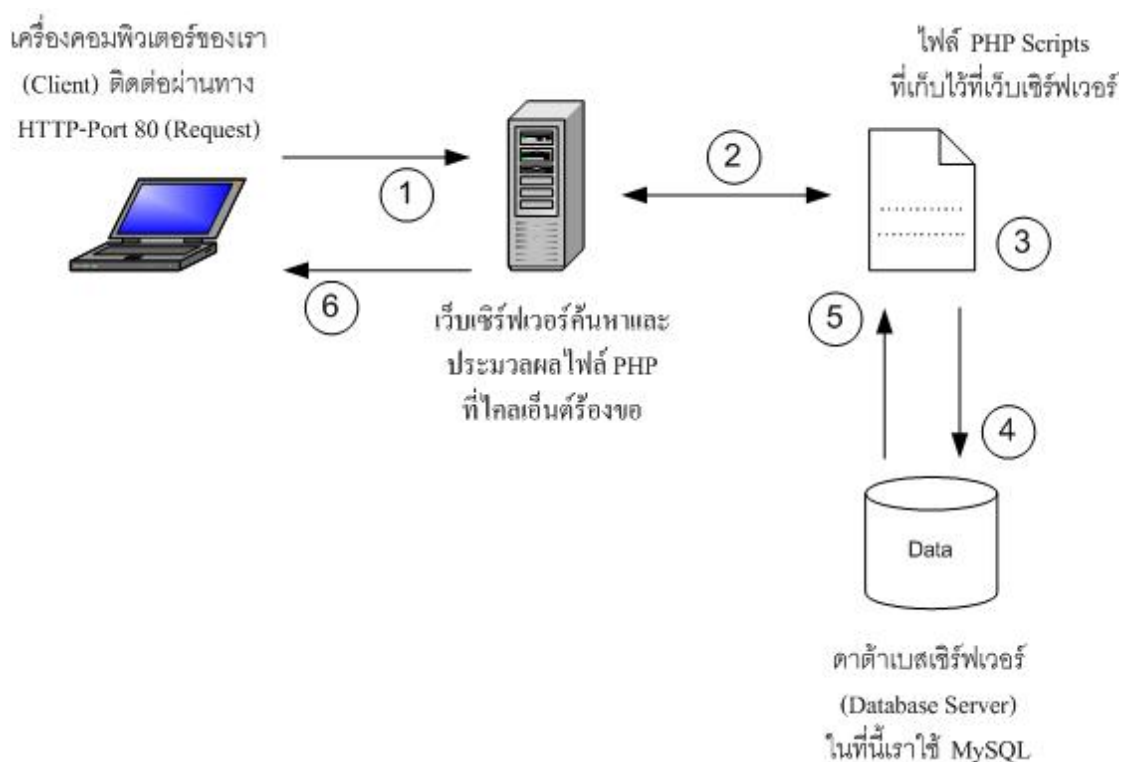
Visual Studio .NET ถือเป็นเครื่องมือพัฒนาแอปพลิเคชันตัวแรกที่จะออกมารับการทำงานตามแนวความคิดของสถาปัตยกรรม .NET Framework ซึ่งประกอบไปด้วยเครื่องมือต่างๆ ดังนี้

- Visual Basic .NET
- Visual C++ .NET
- Visual C# .NET
- ASP .NET

ภาษา Visual C# ถือเป็นภาษาที่เกิดขึ้นมาพร้อมกับแนวความคิดของการเขียนโปรแกรมในยุค .NET อาจจะกล่าวได้ว่า ภาษา Visual C# คือภาษาต้นแบบของการเขียนโปรแกรมใน .NET นั่นเอง โดยที่ภาษาอื่นๆ ที่เกิดขึ้นมาก่อนหน้านี้ จะต้องปรับตัวเข้าหา .NET ทั้งหมด ซึ่งสามารถสังเกตได้ว่าไวยากรณ์การใช้งานของแต่ละภาษานั้น ล้วนแล้วแต่ถูกปรับเปลี่ยนไปจากเวอร์ชันก่อนหน้านี้อย่างสิ้นเชิง

2.2 Web Application

Web application คือ program ที่อยู่ใน web server ที่คอยให้บริการสิ่งที่ร้องขอ (request) จากทาง client ผ่าน protocol HTTP ซึ่งจะแสดงผลที่ร้องขอในรูปของ HTML page ผ่านทาง browser ซึ่งก็คือเว็บไซต์ต่างๆ ที่เราใช้บริการอยู่นั่นเอง Web Application สามารถตอบสนองความคิด Distributed Processing ได้ในระดับหนึ่งซึ่งก็คือ การแบ่งการประมวลผลไว้ที่ฝั่ง Client และฝั่ง Server และมักจะมีการใช้ฐานข้อมูล (database) ควบคู่กับการทำ Web Application ไปด้วยตามความต้องการในการทำ E-Business และ E-Commerce ที่กำลังเป็นที่นิยมในปัจจุบัน แต่ปัญหาที่ตามมาคือ เรื่องของการจ่ายเงินหรือที่เรียกว่า E-payment หรือ Payment-Gateway ซึ่ง Web Application ที่ทำ E-Commerce ต้องใช้บริการจากธนาคาร Online ในการจัดเก็บเงินกับลูกค้า ด้วยเทคโนโลยีนี้ การใช้บริการเก็บเงินจากธนาคารออนไลน์ ซึ่งมีความจำเป็นที่ผู้ค้าจะต้องไปทำการตกลงกับธนาคารและเขียนโปรแกรมให้ตรงตามมาตรฐานที่ธนาคาร Online กำหนดไว้



รูปที่ 2.2 แสดงการเชื่อมต่อ Database Server กับ Web Server

การทำ Web Application ก่อนอื่นต้องติดตั้งเว็บเซิร์ฟเวอร์ขึ้นมา และมี DBMS (Database Management System) ทำหน้าที่จัดการข้อมูลเกี่ยวกับฐานข้อมูลทั้งหมด เครื่องมือ (Tools) ที่นำมาใช้เป็นเว็บเซิร์ฟเวอร์ มีให้เลือกได้หลายตัว ได้แก่

- IIS (Internet Information Server)
- Apache บน LINUX, UNIX

- Oracle

จากรูปจะเห็นการทำงานของ Web Application ซึ่งประกอบไปด้วยส่วนประกอบต่างๆ ซึ่งอาจจะมากกว่านี้ก็ได้แล้วแต่ Application ซึ่งขั้นตอนคร่าวๆจะสรุปเป็นขั้นตอนต่างๆ ดังนี้

ขั้นตอนที่ 1 ผู้ไคลเอนต์ (Client) จะทำการร้องขอหรือเรียกใช้งานไฟล์ PHP ที่เก็บในเครื่องเซิร์ฟเวอร์ (Server)

ขั้นตอนที่ 2 ผู้เซิร์ฟเวอร์จะทำการค้นหาไฟล์ PHP แล้วทำการประมวลผลไฟล์ PHP ตามที่ไคลเอนต์ทำการร้องขอมา

ขั้นตอนที่ 3 ทำการประมวลผลไฟล์ PHP

ขั้นตอนที่ 4 และ 5 เป็นการติดต่อกับฐานข้อมูล และนำข้อมูลในฐานข้อมูลมาใช้ร่วมกับการประมวลผล

ขั้นตอนที่ 6 ส่งผลลัพธ์จากการประมวลผลไปให้เครื่องไคลเอนต์

เครื่องมือที่ใช้เป็น DBMS ก็มีให้เลือกหลายตัวเช่น SQLServer, ORACLE, INFORMIX, ODBC, SYBASE, PHPMyAdmin และ DB2 เป็นต้น แต่ในที่นี้ใช้ MySQL ควบคู่ไปกับ phpMyAdmin

MySQL : MySQL เป็น Database Server ที่เหมาะกับองค์กรขนาดกลางที่มีข้อมูลไม่มากนัก และเป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System) ซึ่งเป็นฟรีแวร์ทางด้านฐานข้อมูล จึงได้รับความนิยมอย่างมากในปัจจุบัน โดยสามารถดาวน์โหลดซอร์สโค้ด (Source Code) ได้จากอินเทอร์เน็ตโดยไม่ต้องเสียค่าใช้จ่าย และสามารถแก้ไขได้ตามความต้องการ พร้อมทั้งยังสนับสนุนการใช้งานบนระบบปฏิบัติการ ตัวอย่างเช่น Unix, Mac และ Windows นอกจากนี้ยังทำงานร่วมกับ Java, C, C++, PHP, ASP หรือ Perl ได้

phpMyAdmin : การทำงานของ MySQL นั้นอาจจะยุ่งยากในการคีย์ข้อมูล เพราะทำงานบน DOS ตัว phpMyAdmin ซึ่งเป็นโปรแกรม mysql client แบบ open source ที่ใช้จัดการ mySQL ได้ผ่านหน้า web browser โดยโปรแกรมนี้เขียนด้วย PHP

2.3 Web Service

ปัจจุบันท่ามกลางสถานะเศรษฐกิจที่เปลี่ยนแปลง ประกอบกับการแข่งขันทางธุรกิจที่สูงขึ้น ปัจจัยหลายๆ อย่างเข้ามามีอิทธิพลต่อรูปแบบการดำเนินธุรกิจ ทว่าแนวโน้มที่เห็นได้ชัดเจนสำหรับธุรกิจในยุคใหม่ คือ การทำธุรกิจในลักษณะอิเล็กทรอนิกส์หรือที่เรียกว่า พาณิชยกรรมอิเล็กทรอนิกส์ (E-Business) ซึ่งเป็นวิธีการทำธุรกิจที่น่าสนใจและกำลังแพร่หลายในหลายๆ ธุรกิจ รวมทั้งยังเป็นปัจจัยหนึ่งที่มีส่วนทำให้องค์กรทางธุรกิจนั้นๆ ยังยืนหยัดอยู่ได้ องค์กรทาง

ธุรกิจต่างๆ ได้เรียนรู้ที่จะนำข้อมูลและทรัพยากรด้านเทคโนโลยีสารสนเทศ มาใช้เป็นเครื่องมือในการพัฒนาธุรกิจของตนมากยิ่งขึ้น เพื่อเป็นฐานที่มั่นคงและการเติบโตทางเศรษฐกิจในอนาคต

องค์กรทางธุรกิจต่างๆ เริ่มมองเห็นวิธีที่ดีที่สุดสำหรับใช้ในการพัฒนาประสิทธิภาพของการปฏิบัติงาน ลดค่าใช้จ่าย และสานสัมพันธ์ที่มีกับลูกค้ารวมถึงซัพพลายเออร์รายสำคัญให้แน่นแฟ้นยิ่งขึ้น โดย สร้างวิธีการดำเนินธุรกิจแบบใหม่ที่ใช้เทคโนโลยีผสมผสานกับเครือข่ายอินเทอร์เน็ตที่แพร่หลายในปัจจุบัน ซึ่งเทคโนโลยีที่กำลังได้รับความสนใจและยังเป็นเครื่องมือที่สนับสนุนความต้องการดังกล่าวในธุรกิจยุคใหม่ก็คือ **Web Services**

เป็นที่ยอมรับกันว่าเทคโนโลยีสารสนเทศและเครือข่ายอินเทอร์เน็ต เป็นเครื่องมือธุรกิจที่จำเป็นในการดำเนินธุรกิจยุคใหม่ไปแล้ว ซึ่ง Web Application เป็นพระเอกหรือเป็นเครื่องมือสำคัญในการทำธุรกิจบนโลกของอินเทอร์เน็ต แต่ปัจจุบัน Web Application กำลังจะกลายเป็นเพียงตัวประกอบ ด้วยการที่องค์กรธุรกิจเริ่มหันมาสนใจเทคโนโลยีใหม่อย่าง Web Services ซึ่งเข้ามาช่วยเพิ่มประสิทธิภาพให้กับระบบการดำเนินธุรกิจขององค์กร และยังสามารถทำงานร่วมกับ Web Application เดิมได้เป็นอย่างดี เพื่อให้องค์กรธุรกิจต่างๆ ได้เห็นแนวทางในการนำ Web Services ไปใช้ในธุรกิจของตน แต่ละองค์กรจำเป็นต้องเข้าใจความหมายของ Web Application และ Web Services เพื่อที่จะได้เห็นความแตกต่างและนำไปประยุกต์ใช้งานได้อย่างถูกต้องและมีประสิทธิภาพมากที่สุด

2.3.1 Web Service คืออะไร

หากเรามองย้อนกลับไปซักไม่กี่ปีที่ผ่านมาใครจะไปคาดคิดว่าเว็บมันจะเติบโตและได้รับความนิยม สูงมากขนาดนี้ ทุกวันนี้หลายๆ คนคงขาดเว็บไม่ได้ เหตุผลที่เว็บประสบความสำเร็จก็คงเป็นเพราะเหตุผลเพียงไม่กี่อย่างคือ ความสะดวก และใช้งานง่าย ในฝั่งผู้ให้บริการ (ผ่านเว็บ) ก็จะมองว่าถ้ามีเว็บเซอร์เวอร์ ก็ขายสินค้าได้ทั่วโลก ในฝั่งผู้ใช้งาน ขอให้คุณเลื่อนเมาส์กับใช้ keyboard เป็น คุณก็ติดต่อ ค้นหา ชื่อของ ได้ทั่วโลก ในมุมมองของ Software เว็บก็ทำหน้าที่อยู่ 3 อย่างคือ GET POST และ PUT ในเรื่องของ Web Service ก็คือการใช้ Web ที่ไม่เพียงแต่เกี่ยวกับข้อมูลอย่างเดียว แต่หมายถึงการบริการด้วย

คำว่า Service ไม่ได้หมายถึงอะไรที่เด่นชัดอย่าง Promool.com Pantip.com แต่หมายถึงส่วนประกอบที่คนอื่น ๆ นำไปใช้ในการให้บริการที่กว้างกว่านี้ด้วย ตัวอย่างเช่น Microsoft Passport ที่ให้บริการตรวจสอบความเป็นตัวตนจริง (Authentication) ผ่านเว็บ ทำให้การบริการข่าวของ Bangkok Post ไม่ต้องตรวจสอบการเข้าสู่ระบบเอง แต่ยกให้ Passport เป็นตัวจัดการแทน หรืออย่าง Dynamic services whitepaper ของ Oracle ก็มีหน้าที่ให้บริการ แปลงค่าเงิน แปลงภาษา การส่งของ กระบวนการเคลมสินค้า เป็นต้น ส่วนความหมายอย่างเป็นทางการของ Web Service ก็คงเป็นของ IBM ที่กล่าวว่า

เว็บเซอวิส คือ Web Application ยุคใหม่ ที่ประกอบด้วยส่วนย่อยๆ มีความสมบูรณ์ในตัวเอง สามารถติดตั้ง ค้นหา เริ่มทำงานได้ผ่านเว็บ Web Service สามารถทำอะไรก็ได้ตั้งแต่งานง่ายๆ เช่นดึงข้อมูล จนถึงกระบวนการทางธุรกิจที่ซับซ้อน เมื่อ Web Service ตัวใดตัวหนึ่งเริ่มทำงาน Web Service ตัวอื่นก็สามารถรับรู้และเริ่มทำงานได้อีกด้วย

หลายคนอาจจะถามว่าทำไมต้องเป็น Web เพราะเรามี Middle Ware อื่นๆ มากมายเช่น RMI Jini CORBA DCOM ฯลฯ แม้ Middle Ware เหล่านี้จะสามารถรองรับได้ แต่ไม่มีตัวใดตัวหนึ่งที่เด่นจริง แต่ในเมื่อ Web มีจุดเด่นในเรื่องของการให้บริการข้อมูลที่สะดวก ใช้งานง่าย จึงกลายเป็นตัวประสาน Middle Ware ต่างๆ เข้าด้วยกันซึ่งจะให้คุณกันเองคงยาก ยิ่ง Web ทำหน้าที่เป็นตัวกลางให้ Middle Ware เหล่านี้สามารถคุยกันได้ และมีประสิทธิภาพกว่าวิธีการเดิมๆ มาก

หากเรามองจากกรณีของ n-tier application จะพบว่า web service คือกลไกในการเข้าถึงบริการที่แต่ละ Middle Ware ให้บริการ การเข้าถึงจะอาศัย Listener และส่วนประกอบที่ระบุถึงบริการต่างๆ ที่รองรับการทำงาน โดยการทำงานจริงๆ นั้นก็ใช้วิธีการปกติของ Middle Ware นั้นๆ

2.3.2 พื้นฐานของ Web Service

พื้นฐานของ Web Service ก็คือ XML กับ HTTP ซึ่งจะพบว่า HTTP ก็เป็นที่รู้จักกันดี และไปได้ทั่วทุกแห่งที่มี internet ส่วน XML ก็ภาษาสากลที่คุณสามารถปรับแต่งได้ตามใจชอบ เพื่อให้เกิดกิจกรรมระหว่าง client และบริการ หรือระหว่างส่วนประกอบต่างๆ เบื้องหลัง Web server ก็คือ ข้อความ XML จะถูกแปลงให้การขอบริการจาก Middle ware และผลที่ได้ก็จะแปลงกลับมาในรูป XML

ยกตัวอย่างให้เห็นง่ายๆ คุณต้องการให้เครื่อง PC อ่านค่าจาก serial port แล้วส่งไปประมวลผลบนเครื่อง UNIX แล้วส่งผลกลับมาแสดงบนจอ PC ถ้าเป็นเมื่อก่อน คุณก็ต้องแปลงข้อมูลที่ได้อี้อยู่ในรูปของ ASCII แล้วส่งไปยัง UNIX พร้อมคำสั่งว่าให้ทำอะไร ในฝั่ง UNIX คุณก็ต้องมาแยกว่าอันไหนคือคำสั่ง อันไหนคือข้อมูล เมื่อประมวลผลแล้ว จะส่งกลับมาในรูปแบบไหน แล้วถ้าหากจะส่งไปหาเครื่องที่เป็น MAC ท่านจะต้องเขียนโปรแกรมเพิ่มในส่วนไหนบ้าง จะพบว่าเราต้องพัฒนากันเป็นคู่ๆ ไป และต้องนิยามในแต่ละฝั่งให้ชัดเจน แต่หากเป็น Web Service คุณจะพบว่า เราแปลงข้อมูลให้อยู่ในรูป XML แต่ละคุณก็ต้องการรู้แค่ มาตรฐาน XML ก็พอ แล้วต่างคนต่างก็เขียน Service ของตัวเอง ไม่ต้องกังวลเรื่องของการเชื่อมโยงอีกต่อไป และ Protocol ที่ส่งก็คือ HTTP นั่นเอง ถ้าท่านเชื่อมโยงกับ HTTP (หรือเว็บ) ได้ ท่านก็ใช้บริการทุกอย่างได้

แต่เดี๋ยวก่อนการเข้าถึงและการส่งงานนั้นยังเป็นเพียงโครงสร้างพื้นฐาน แต่ในความเป็นจริงยังมีอะไรมากกว่านั้น เช่น การค้นหา การทำธุรกรรม ความปลอดภัย การพิสูจน์ตัวตน และอื่นๆ อันเป็นบริการที่ทำให้เป็นบริการพื้นฐานจริงๆ

ระบบเพิ่มเติมที่ต้องมีและต้องรักษาความสะดวกและใช้งานง่ายไว้ด้วย พื้นฐานของ Web Service เต็มรูปแบบคือ XML + HTTP + SOAP + WSDL + UDDI หรือในระดับสูงกว่านั้น แต่ไม่ได้ถือเป็นสิ่งจำเป็นเสมอไปก็ต้องเพิ่มเทคโนโลยี XAML, XLANG, XKMS, XFS เป็นต้น

2.3.3 เทคโนโลยีที่ใช้ในการพัฒนาเว็บเซอร์วิส

การทำงานของ Services ประกอบไปด้วย มาตรฐานหลัก 4 อย่าง ซึ่งสามารถอธิบายอย่างง่ายๆ ได้ดังนี้

XML (Extensible Markup Language)

เป็นภาษา Markup ที่เป็น text-based ซึ่งทำให้เป็น มาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตอย่างรวดเร็ว ผู้ที่ทำหน้าที่รับผิดชอบ และกำหนดมาตรฐานของ XML คือ World Wide Web Consortium (W3C) ความแตกต่างระหว่าง XML กับ HTML คือ HTML ถูกนำมาใช้ในการสร้าง เว็บเพจ ที่สามารถแสดงผลได้โดยโปรแกรมเบราว์เซอร์ แต่ XML จะใส่ tags ได้อย่างอิสระ แล้วทำการส่ง XML ชุดนี้ไป ประมวลผลยังแอปพลิเคชันใด ๆ ที่สามารถใช้ข้อมูลใน XML นี้

XML & HTML

HTML ภาษาที่ใช้ในการเขียน Web มากที่สุดนั้นเป็นเพราะมีรูปแบบที่ง่ายต่อการแสดงผลของ Browser เนื่องจาก มี tag ตายตัวที่สามารถบอกได้ว่าเมื่อเจอ tag นี้จะแสดงผลอย่างไร เช่น เมื่อเจอ tag ... ในเอกสารก็ให้แสดงข้อความที่อยู่ระหว่าง tag เป็นตัวหนา แต่จะสังเกตได้ว่าคอมพิวเตอร์จะไม่เข้าใจว่าข้อความนั้นคืออะไร เพียงแต่รู้ว่าแสดงผลอย่างไร นั่นแสดงว่าไม่สามารถนำข้อมูลภายใน tag เหล่านี้ไปทำการประมวลผลใดๆ ได้เลย

XML เป็นภาษาที่มีลักษณะเป็น tag คล้าย HTML แต่ไม่ได้มุ่งที่การแสดงผล XML มุ่งที่การสื่อความหมายโดยอนุญาตให้ผู้ใช้สามารถกำหนด tag ขึ้นได้เองเพื่อให้สื่อความหมายทางภาษาของมนุษย์ แต่คอมพิวเตอร์เองก็เข้าใจเช่นกัน ทำให้ข้อมูลระหว่าง tag สามารถนำไปประมวลผลต่อได้ เช่น

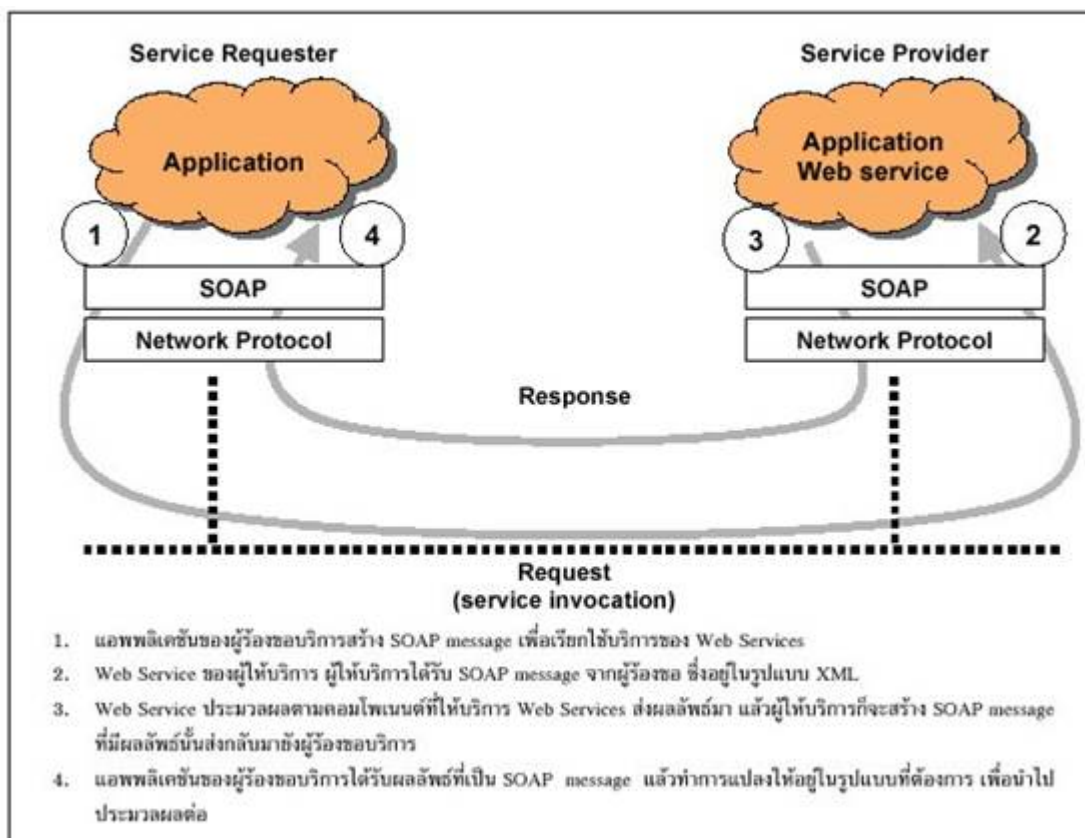
```
<ComputerBook>
  <book>
    <name>เว็บเซอร์วิส</name>
    <price>10.00$</price>
  </book>
  <book>
    <name>xml</name>
    <price>10.00$</price>
  </book>
</computerBook>
```

จากตัวอย่าง จะเห็นได้ว่ารูปแบบเอกสาร xml ไม่ได้บอกวิธีแสดงผลไว้ แต่เอกสารสามารถสื่อความหมายให้คอมพิวเตอร์เข้าใจได้ และนำไปประมวลผลต่อได้ แต่ถ้าเราต้องการจะแสดงผลเราต้องใช้ควบคู่กันกับ HTML ข้อมูลที่มีโครงสร้างของภาษา XML จะถูกนำไปประมวลผลต่ออย่างอัตโนมัติได้อย่างง่ายดาย ภาษา XML จึงถูกนำมาใช้เป็นภาษามาตรฐานในการแลกเปลี่ยนข้อมูลของ Web Services

SOAP (Simple Object Access Protocol)

SOAP กลายเป็นสิ่งที่มีความสำคัญสำหรับ Web Services อย่างรวดเร็ว เป็นโปรโตคอลที่ผู้จัดหา Web Services เลือกใช้ที่จะส่ง message ระหว่าง Web Services SOAP เป็น Transport Protocol ที่มี XML เป็นพื้นฐานและใช้ HTTP เป็นโปรโตคอลร่วมในการส่งผ่านเครือข่าย SOAP จะระบุวิธีในการเข้ารหัสส่วนหัว (Header Encoding) ของทั้ง HTTP และไฟล์ XML ไว้อย่างชัดเจนทั้งใน ส่วนของการติดต่อไปยังคอมพิวเตอร์เครื่องหนึ่งและส่งผ่านข้อมูลไปให้ รวมถึงระบุวิธีที่โปรแกรมซึ่งถูกเรียกนั้นจะส่งค่าคืนกลับมาด้วย

SOAP (Simple Object Access Protocol) เป็น XML-based โปรโตคอล (lightweight protocol) และใช้ HTTP เป็นโปรโตคอลร่วม สำหรับ การแลกเปลี่ยนข้อมูลในสภาวะแวดล้อมแบบกระจายศูนย์ (decentralized, distributed environment) SOAP ได้ กำหนดเมสเสจจิงโปรโตคอล (Messaging Protocol) ระหว่างผู้ขอบริการ (requestor) กับผู้ให้บริการ (provider) เช่น ผู้ขอบริการสามารถติดต่อแลกเปลี่ยนข้อมูลกับผู้ให้บริการโดยใช้ RMI (Remote Method Invocation) ตามวิธีการของ โปรแกรมแบบออปเจ็ค บริษัทไมโครซอฟท์, ไอบีเอ็ม, โลตัส, ยูสเซอร์แลนด์ (UserLand) และ ดีเวลลอปเปอร์เมนเตอร์ (DeveloperMenter) ได้ร่วมกันกำหนดมาตรฐานของ SOAP ขึ้น ซึ่งต่อมาได้มีบริษัทอีก 30 กว่าบริษัทเข้าร่วมและ จัดตั้งเป็น W3C XML Protocol Workgroup ขึ้น SOAP ได้กำหนดรูปแบบพื้นฐานของการสื่อสารแบบกระจายขึ้นโดยการพัฒนา SOAP แม้ว่า SOAP จะไม่ได้กำหนดเมสเสจจิงโปรโตคอล (Messaging Protocol) ไว้ แต่ SOAP ได้ถูกกำหนด ให้เป็น Services-Oriented Architecture Protocol เรียบร้อยแล้ว เนื่องจากมันได้ถูกใช้ในการพัฒนา SOA อย่างแพร่ หลายแล้วนั่นเอง จุดเด่นของ SOAP ก็คือเป็นโปรโตคอลที่เป็นกลาง กล่าวคือ ไม่มีใครเป็นเจ้าของและเป็นโปรโตคอลที่ทำงานกับโปรโตคอลอื่นหลายชนิด การพัฒนาก็อนุญาตให้ทำได้อย่างอิสระตามแพลตฟอร์มระบบปฏิบัติการแบบจำลองทางวัตถุ (Object model) และภาษาโปรแกรมของผู้ที่ทำการพัฒนา



รูปที่ 2.3 XML message using SOAP

ทำไมต้องเป็น SOAP

เนื่องจากจุดประสงค์หลักของการใช้งานเว็บเซอร์วิส เราต้องการให้แอปพลิเคชัน มีการทำงานกับแอปพลิเคชันที่ทำงานอยู่ในเครื่องอื่นโดยผ่านทางเครือข่าย ซึ่งเทคโนโลยีที่มีอยู่ในปัจจุบันที่ใช้มีการสื่อสารระหว่าง Object ในระยะไกล (Remote Procedure Calls : RPC) เช่น DCOM,EJB หรือ CORBANั้นไม่ได้ถูกออกแบบมาใช้สำหรับโปรโตคอล HTTP (เครือข่ายที่ปัจจุบันใช้งานแพร่หลาย และสะดวกมากที่สุดได้แก่เครือข่าย Internet ซึ่งให้การสื่อสารผ่านโปรโตคอล HTTP เป็นส่วนใหญ่)

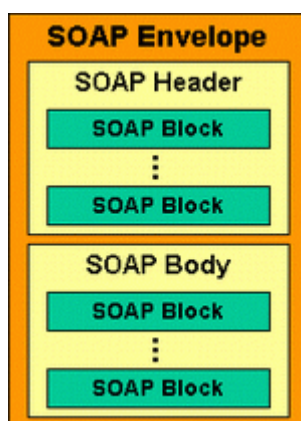
เทคนิค RPC ของเทคโนโลยีที่กล่าวข้างต้นนั้นต่างก็มีปัญหาในด้านการนำมาใช้งานในแง่ของความเข้ากันได้ของการเรียกใช้งานข้ามเทคโนโลยี เนื่องจากเป็นเทคโนโลยีเฉพาะของแต่ละค่าย ยกเว้น CORBA ผู้พัฒนาระบบจะต้องพัฒนาโปรแกรมที่มีความซับซ้อน และยังมีปัญหาในส่วนของ Firewalls และ Proxy Server ด้วยเนื่องจากโดยปกติ เซอร์เวอร์จะปิดการสื่อสารที่ไม่ใช่โปรโตคอล HTTP ออกไป เพื่อความปลอดภัยของระบบที่มีการติดต่อสื่อสารกับภายนอก

ดังนั้นทางเลือกของการสื่อสารที่จะนำมาใช้ในการให้บริการเว็บเซอร์วิส ก็คือให้ทำงานอยู่บน โปรโตคอล HTTP ไปเลย ซึ่ง SOAP นอกจากจะทำงานบนโปรโตคอล HTTP แล้วยังเป็นมาตรฐานเปิดที่จะทำให้สามารถติดต่อสื่อสารกับเครื่องคอมพิวเตอร์ที่มีความแตกต่างกันทั้งระบบปฏิบัติการ, เทคโนโลยี รวมไปถึงภาษาที่ใช้ในการพัฒนาด้วยก็ได้

โครงสร้างของ SOAP

เอกสาร SOAP นั้นมีโครงสร้างในรูปแบบ XML ซึ่งเราสามารถแบ่งเป็นส่วนของเอกสารได้เป็น 3 ส่วนหลักดังนี้คือ

1. SOAP envelop เนื้อหาสาระ (Content) ของเอกสารทั้งหมด
2. SOAP header ส่วนเพิ่มเติมของเอกสาร SOAP ซึ่งจะมีก็ได้ หรือไม่มีก็ได้
3. SOAP body ส่วนที่ใช้ในการเรียกใช้งานเซอร์วิส และผลลัพธ์ที่ได้จากเซอร์วิส



รูปที่ 2.4 โครงสร้างของเอกสาร SOAP

ตัวอย่างเอกสาร SOAP อย่างง่ายของการสอบถามราคา ดอกกุหลาบ จำนวน 100 ดอก

```
<soap:Envelope>
  <soap:Body>
    <GetPrice>
      <Item>Rose</Item>
      <Quantity>100</ Quantity >
    </ GetPrice >
  </ soap:Body >
</ soap:Envelope >
```

เรามาดูอีกตัวอย่างหนึ่งของเอกสาร SOAP ที่จะแสดงในรูปด้านล่างเพื่อจะได้ทำความเข้าใจในส่วนต่างๆ ที่มีความซับซ้อนขึ้น แต่ก็อย่าจนเกินไปที่จะทำความเข้าใจได้ดังรูปที่ 2 SOAP Envelope ต่อไปนี้

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <soap:Header>
    <!--ข้อมูลในส่วนของ Header -->
    <i:local xmlns:i="http://www.i3t.or.th/ws/">
      <i:currency>Bath</i:currency>
    </i:local>
  </ soap:Header>
  </ soap:Body>
    <!--ข้อมูลในส่วนของ Body -->
    <GetPrice>
      <Item>Rose</Item>
      <Quantity>100</ Quantity >
    </ GetPrice >
  </ soap:Body >
  <soap:Fault>
    <!--ข้อมูลของ SOAP ในการมีข้อผิดพลาด จาก SOAP Node -->
  </soap:Fault>
</ soap:Envelope >

```

คำอธิบาย

<Envelope>

จะเป็น Root ของเอกสาร XML SOAP เสมอ โดยจากตัวอย่างจะใช้ Namespaces จาก "http://schemas.xmlsoap.org/soap/envelope/" (Schema for the SOAP/1.1 envelope) ดังนั้น Element ต่างๆ ในตัวอย่างนี้ที่อยู่ในเอกสาร XML ส่วนที่เป็นมาตรฐานของ SOAP เราจะมี Namespaces ที่เรารวบรวมเป็น soap (<soap:Envelop>, <soap:Header>, <soap:Body>, <soap:Fault>)

Attribute ใน Envelop

Attribute encodingStyle ในตัวอย่างมีการอ้างการ encoding จาก

"http://schemas.xmlsoap.org/

soap/encoding/" (Schema for the SOAP/1.1 encoding) จะใช้ในกรณีที่เรามีการใช้ Parameter หลายชนิดข้อมูลในเอกสาร (Data Type) เช่น Boolean, String, Integer เป็นต้น

<Header>

จากตัวอย่างมีการเพิ่มเนื้อหาของเอกสาร SOAP เข้าไปในส่วนของ Header ในที่นี้คือ สกุลเงิน (Currency) ซึ่งส่วนนี้จะใช้ในส่วนของการ Application ไม่ได้เป็นส่วนหนึ่งของมาตรฐานของ SOAP แต่ผู้ใช้งานเป็นผู้กำหนดใช้เอง (User-defined)

Attribute ใน Header

ใน <Header> Element อาจจะมีการใส่ Attribute mustUnderstand เพื่อให้ฝั่งที่รับเอกสาร SOAP ให้ทำการประมวลผลในส่วน Header ด้วยโดยกำหนดค่าเป็น Boolean เช่น

```
<i:local xmlns:l="http://www.i3t.or.th/ws/">
  <i:country mustUnderstand="1">Thailand</i:country>
  <i:currency mustUnderstand="0">Bath</i:currency>
</i:local>
```

ค่าของ mustUnderstand ถ้าเป็น "0" แสดงว่าไม่ต้องประมวลผล ถ้าเป็น "1" : true คือ จำเป็นต้องมีการประมวลผล (โดยค่า default = "0" : fault)

<Body>

โดยปกติแล้วเอกสาร SOAP จะต้องมีส่วนของ Body เพราะเป็นส่วนเป็นเนื้อหาสาระจริงๆ ของ SOAP จากตัวอย่างเราต้องการสอบถามราคาของดอกกุหลาบจำนวน 100 ดอก ซึ่ง <GetPrice>, <Item> และ <Quantity> เป็น Element ที่ใช้งานใน Application ไม่ใช่มาตรฐานของ SOAP

<Fault>

เป็นส่วนที่จะถูกใช้เมื่อมีข้อผิดพลาดในการประมวลผลของเอกสาร SOAP ซึ่งโดยปกติจะเห็นเฉพาะที่เป็นเอกสารตอบกลับเท่านั้น (Reply Message หรือ Answer Message) ซึ่งตัวอย่างจะเป็นเอกสารที่เป็นการร้องขอบริการ (Request Message) ซึ่งไม่น่าจะมีส่วนของ <Fault> Element แต่ที่เขียนไว้เพื่อให้เห็นภาพรวมของโครงสร้างเอกสาร SOAP ทั้งหมดก่อน ตัวอย่างข้อมูลใน <Fault> Element ที่มีการเกิดข้อผิดพลาด

```
<soup:Fault>
  <faultcode>Client</faultcode>
  <faultstring>Invalid Request</faultstring>
</soap:Fault>
```

ตารางอธิบาย Fault Element

Sub Element	คำอธิบาย
<faultcode>	ชื่อของการเกิดข้อผิดพลาด (ดูตารางด้านล่าง)
<faultstring>	คำอธิบายของข้อผิดพลาด
<faultactor>	ชื่อของแหล่งที่เกิดข้อผิดพลาด
<detail>	ข้อมูลรายละเอียดเพิ่มเติมของข้อผิดพลาดนั้น

จะเห็นว่า SOAP Envelope ก็เป็นเอกสารที่อยู่ในรูปแบบ XML ทั่วไป แต่สิ่งที่ทำให้ SOAP มีความสามารถมากขึ้นก็คือ SOAP ได้มีการกำหนดโครงสร้างของเอกสารเป็นส่วนๆ โดยอธิบายว่าส่วนใดมีหน้าที่อะไร (What) มีข้อมูลอะไรอยู่ในส่วนนั้น และใคร (Who) คือผู้ที่จะต้องสนใจในส่วนนั้น ๆ และกล่าวถึงว่าส่วนใดจำเป็นที่ต้องมีในเอกสาร และส่วนใดเป็นส่วนที่เพิ่มเติมอาจจะมีหรือไม่มีส่วนนั้นก็ได้ขึ้นอยู่กับความจำเป็นในการใช้งาน ทำให้การพัฒนาโปรแกรมมีทิศทางที่ค่อนข้างชัดเจน แต่สามารถเพิ่มขยายได้ในอนาคตอีกด้วย

SOAP Intermediaries

จากตัวอย่างเอกสาร SOAP นั้นจะเห็นได้ว่าในส่วนของ <Header> Element จะเป็นส่วนที่เราจะขยายสารสนเทศ (information) ใหม่ๆ เข้าไปใน SOAP ได้ซึ่งตัวอย่างเรามีการอธิบายให้ผู้รับเอกสารตัวอย่างนี้ให้ทราบว่าในการสอบถามราคานี้ให้ใช้เงินในสกุลเงินบาท เป็นต้น ซึ่งเราจะเรียกการขยายในส่วนนี้ว่าเป็นการขยายในแนวดิ่ง (Vertical extensibility) แต่ถ้าเราจะมาดูในด้านการขยายในแนวราบ (Horizontal extensibility) ของ SOAP นั้นก็คือทำอะไรที่จะให้มีการส่งเอกสาร SOAP ไปยังปลายทางโดยเส้นทางของการเดินทางของเอกสารอาจจะไม่เป็นลักษณะแบบจุดต่อจุด (Point-to-point) แต่อาจจะเป็นการส่งเอกสารตามเส้นทางโดยเอกสารยังคงเป็นเอกสารตัวเดิม

ความจำเป็นในการใช้ SOAP Intermediary

SOAP Intermediaries จะทำหน้าที่เป็นตัวผ่าน หรือตัวกลางในการส่งเอกสาร SOAP จากต้นทาง ในที่นี้คือ Requestor ไปยัง Provider เพื่อสร้างกระบวนการบางอย่างกับเอกสาร ซึ่งในการสร้างเส้นทางในการส่งแบบนี้อาจมีจุดประสงค์ดังต่อไปนี้คือ

1. **Intermediary** ทำหน้าที่เป็น Crossing trust domains เนื่องจากการทำงานของ Web Services เป็นการเรียกใช้งานแบบระยะไกล หรือมีการทำงานแบบระบบกระจาย (Distributed System) ดังนั้นวิธีการในด้านความปลอดภัยจึงเป็นส่วนหนึ่งที่ต้องคำนึงถึง ซึ่งวิธีการหนึ่งก็คือให้มีการส่งเอกสารมายังจุดหนึ่งที่มีความน่าเชื่อถือได้ก่อน แล้วส่งต่อไปยังปลายทางอีกที่หนึ่ง ซึ่งหลักการใช้ Trust Domains เราอาจจะดูตัวอย่างง่ายๆ เช่น ในองค์กรเราอาจจะมีเครือข่าย Intranet อยู่ภายในองค์กร เครื่องคอมพิวเตอร์แต่ละเครื่องอาจจะสามารถติดต่อไปยัง Server ได้โดยตรง แต่ถ้าต้องการออกไปยัง Internet หรือ Extranet ที่อยู่นอกองค์กรจะต้องผ่าน Trust Domains ก่อน ในที่นี้อาจจะเป็น Firewalls หรือ Virtual Private Network (VPN) gateways เพื่อความปลอดภัย

2. **Intermediary** ทำหน้าที่เป็นตัวขยายการทำงานแบบระบบกระจาย (Distributed System) ถ้าเรามองดูเฉพาะแง่ของการขอใช้บริการจาก Requestor ไปยังผู้ให้บริการ Provider โดยผู้ให้บริการจะมีการกระบวนการทำงานบางอย่างภายในและตอบกลับไปยังผู้ใช้ ซึ่งการทำงานแบบนี้ก็อยู่บนพื้นฐานของ Request and Respond นั่นเอง แต่ถ้าระบบของเรามีการขยายใหญ่ขึ้น

สิ่งที่เราอาจจะต้องดูเพิ่มขึ้นก็คือความสามารถในการรองรับงานของผู้ให้บริการด้วย ดังนั้นก่อนที่จะมีการส่งเอกสาร SOAP ไปยังเซิร์ฟเวอร์ ของผู้ให้บริการนั้นอาจจะมีการผ่าน Intermediary Server ก่อนเพื่อทำหน้าที่เป็น Buffer รับเอกสาร SOAP มาพักไว้และส่งเอกสารให้ผู้ให้บริการ (Provider) อีกทอดหนึ่ง การทำงานเช่นนี้จะช่วยให้ Server ของ ผู้ให้บริการไม่รับภาระงานมากเกินไปในช่วงเวลาใดเวลาหนึ่งจนอาจทำให้เกิดความเสียหายได้ และ Intermediary อาจจะมีการรวบรวมเอกสารแล้วส่งมาให้ผู้ให้บริการเป็นลักษณะ Batch เพื่อเพิ่ม Performance การใช้งาน Intermediary ลักษณะนี้ผู้ขอใช้บริการอาจจะไม่เห็น Intermediary แต่จะมองเห็นว่าติดต่อกับผู้ให้บริการโดยตรง

3. การใช้งาน **Intermediary** ในลักษณะ Value-added ให้กับระบบ เช่น ใช้ Intermediary ในการเข้ารหัสข้อมูล (Encrypts) และเซ็นลายเซ็นอิเล็กทรอนิกส์ (Digital signs) ในการส่ง และรับ หรืออีกตัวอย่างหนึ่งเช่นต้องการเก็บข้อมูลเพิ่มนำไปวิเคราะห์การทำงานของระบบโดยการรับ-ส่งเอกสารจะผ่าน Intermediary แต่ไม่ได้ทำการประมวลผลเอกสารแต่จะเก็บข้อมูลบางอย่างไว้เท่านั้นเช่นนำไปวิเคราะห์ค่า QoS (Quality of Service) หรือ Bottleneck ของระบบ เป็นต้น

Intermediaries ใน SOAP

ในหัวข้อก่อนหน้านี้ได้อธิบายหลักการของการใช้ Intermediaries ไปแล้ว ดังนั้นในส่วนนี้จะกล่าวถึงว่าจะทราบได้อย่างไรว่าเอกสาร SOAP นั้นใครจะเป็นผู้ประมวลผล (ซึ่งจะอยู่ในส่วนของ <Body> Element ของเอกสาร)

ดังรูปที่แสดงการทำงานของ Intermediaries จะเห็นว่าเอกสาร SOAP จะถูกส่งจากผู้ขอใช้บริการผ่าน Intermediaries เป็นไปจนถึงปลายทางคือผู้ให้บริการ ดังนั้นเอกสาร SOAP ก็ถูกเปิดเผยทั้งหมดกับ Intermediaries ด้วย แต่ส่วนที่ Intermediaries สนใจจะอยู่เฉพาะส่วนที่เป็น Header เท่านั้น โดยใน Header จะมี Attribute หนึ่งเพื่อใช้ในการระบุว่าใครที่ควรจะทำงานกับ Header นี้ ซึ่ง Attribute นั้นก็คือ actor โดยค่าที่ actor จะต้องใส่ก็คือ URI (Universal Resource Indicator) ของ Intermediary นั้นเอง ดังตัวอย่าง

```
<soup:Header>
  <i:Authentication
    xmlns:i="http://www.i3t.or.th/ws/security"
    soap:actor="urn:I3T:Security/GateWay"
    soap:mustUnderstand="1">
    <i:username>ACompany</i:username>
    <i:password>SOAPpath</i:password>
  </i:Authentication>
</soap:Header>
```

ดังนั้นเมื่อ Intermediary รับเอกสาร SOAP โดยถ้ามีชื่อ attribute actor ตรงกับของตัวเอง Intermediary จะเปลี่ยนเนื้อหาใน Header ที่ได้เพื่อส่งต่อไปยัง Intermediary อื่นหรือไปยังผู้

ให้บริการจุดประสงค์หลักที่มีการออกแบบการทำงานแบบนี้ เพื่อลดความซับซ้อนของเอกสารลง นอกจากนี้แล้วยังมี URI ของ attribute actor ชนิดพิเศษอีกหนึ่งค่าคือ "http://schemas.xmlsoap.org/soap/actor/next" เพื่อบอกว่าผู้รับเอกสาร SOAP ในจุดต่อไปจะเป็นผู้ที่ประมวลผลของเอกสาร ซึ่งกระบวนการนี้จะมีประโยชน์อย่างมากในการส่งเอกสาร SOAP เป็นทอดๆ (hop-by-hop)

WSDL (Web Services Description Language)

WSDL (Web Services Description Language) เป็นภาษาที่ใช้อธิบายคุณลักษณะการใช้บริการของ Web Services และวิธีการติดต่อกับ Web Services ความต้องการของนิยามนี้เกี่ยวข้องกับความต้องการของ distributed system ที่จะกำหนด Interface Definition Language(IDL) โดยใช้ภาษา XML, WSDL เกิดจากการรวมแนวคิดของ NASSL (The Network Accessible Service Specification Language), WDS (Well-Defined Services) ของบริษัทไอบีเอ็ม, SDL (The Service Description Language) และ SCL (the SOAP Contract Language) ของบริษัทไมโครซอฟท์ ปัจจุบัน WSDL เป็นภาษา ที่อยู่ในการดูแลของ W3C (World Wide Web Consortium) ซึ่งยังไม่เป็นมาตรฐานที่สมบูรณ์ เวอร์ชันที่ใช้งานอยู่ใน ปัจจุบันคือ WSDL 1.1 (รายละเอียดเพิ่มเติมเกี่ยวกับ WSDL สามารถศึกษาเพิ่มเติมได้ที่ <http://www.w3c.org/TR/wsdl>)

WSDL คือ มาตรฐานสำหรับการประกาศ process ที่จำเป็นในการเรียกใช้เซอร์วิส SOAP (Simple Object Access Potocal)

โครงสร้างเอกสาร WSDL

WSDL เป็นภาษาที่อยู่ในความดูแลขององค์กร W3C (World Wide Web Consortium) version ที่มีอยู่ในปัจจุบัน คือ WSDL 1.1 ในการใช้งานจริง หากเราสร้างบริการ Web Services ก็จะมีเครื่องมือช่วยสร้างเอกสาร WSDL สำหรับ Web Services อย่างอัตโนมัติ จุดภายในเอกสารที่เราควรรู้เกี่ยวกับการติดต่อและเรียกใช้บริการของ Web Services มีจุดที่ควรรู้ ดังนี้

Element	Definition
<portType>	เป็นส่วนที่สำคัญที่สุดใน WSDL element อธิบาย operations ที่ web service มีให้บริการ และ messages ที่เกี่ยวข้อง เทียบได้กับ function library หรือ module หรือ class ในการเขียนโปรแกรม
<operation>	อธิบาย method ที่ให้บริการ Web Services หนึ่งจะมี method จำนวนกี่ method ก็ได้
<message>	อธิบาย data elements ของ operation แต่ละ message อาจมีมากกว่าหนึ่งส่วนเทียบได้กับ parameter ของ function ในการเขียนโปรแกรม
<types>	อธิบายชนิดข้อมูลที่ web service ใช้ เพื่อความเป็นกลาง WSDL ใช้ XML Schema syntax ในการระบุนชนิดข้อมูล

<binding>	อธิบาย format ของ message และ protocol details ในแต่ละ port
<service>	สำหรับ web server จะมี Web Services จำนวนที่บริการก็ได้ และ ชื่อ Web Services ก็เป็นตัวจำแนกและบ่งบอกแต่ละบริการซึ่งห้ามมีซ้ำกัน

ตามทฤษฎีแล้ว ไฟล์เอกสาร WSDL แต่ละไฟล์ สามารถอธิบายคุณลักษณะของบริการ Web Services ได้มากกว่า 1 บริการโดยแต่ละ Web Services จะมี port สื่อสารเฉพาะตัว ซึ่งบ่งบอกไว้ในเอกสาร WSDL อยู่แล้ว

WSDL Example

```

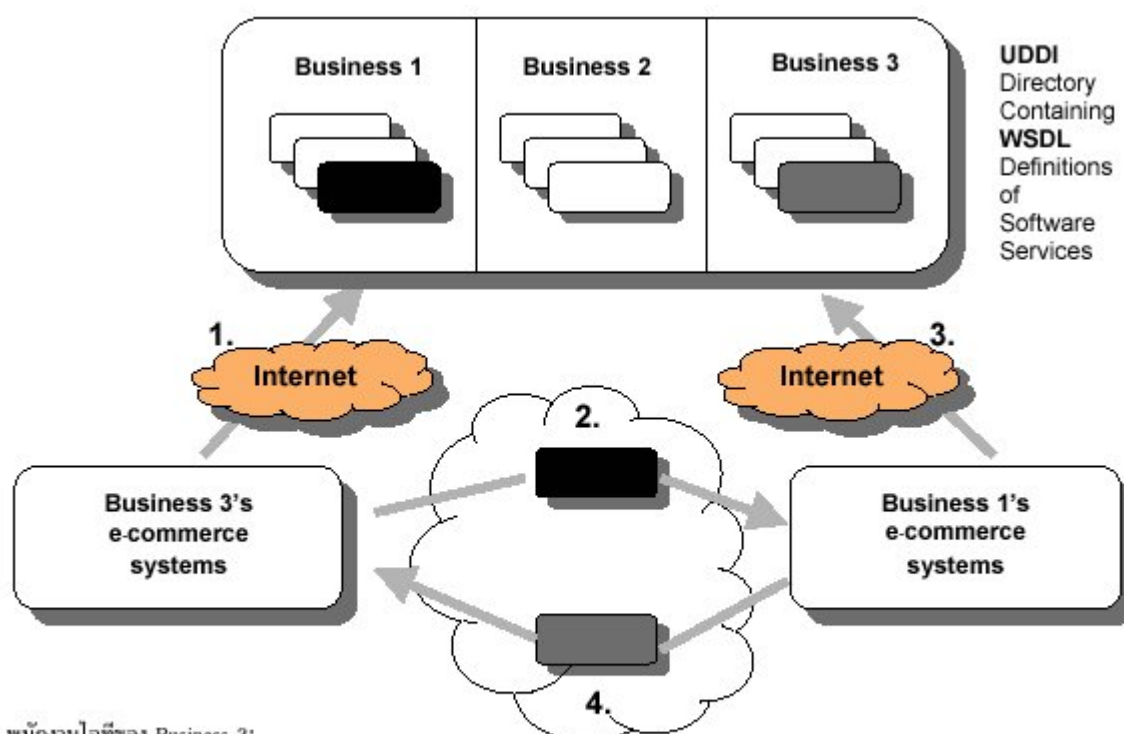
<wsdl:definitions name="nmtoken"? targetNamespace="uri">
  <import namespace="uri" location="uri"/> *
  <wsdl:documentation .... /> ?
  <wsdl:types> ?
    <wsdl:documentation .... /> ?
    <xsd:schema .... /> *
  </wsdl:types>
  <wsdl:message name="ncname"> *
    <wsdl:documentation .... /> ?
    <part name="ncname" element="qname"? type="qname"?/> *
  </wsdl:message>
  <wsdl:portType name="ncname"> *
    <wsdl:documentation .... /> ?
    <wsdl:operation name="ncname"> *
      <wsdl:documentation .... /> ?
      <wsdl:input message="qname"> ?
        <wsdl:documentation .... /> ?
      </wsdl:input>
      <wsdl:output message="qname"> ?
        <wsdl:documentation .... /> ?
      </wsdl:output>
      <wsdl:fault name="ncname" message="qname"> *
        <wsdl:documentation .... /> ?
      </wsdl:fault>
    </wsdl:operation>
  </wsdl:portType>
  <wsdl:serviceType name="ncname"> *
    <wsdl:portType name="qname"/> +
  </wsdl:serviceType>
  <wsdl:binding name="ncname" type="qname"> *
    <wsdl:documentation .... /> ?
    <!-- binding details --> *
    <wsdl:operation name="ncname"> *
      <wsdl:documentation .... /> ?
      <!-- binding details --> *
      <wsdl:input> ?
        <wsdl:documentation .... /> ?
        <!-- binding details -->
      </wsdl:input>
      <wsdl:output> ?
        <wsdl:documentation .... /> ?
        <!-- binding details --> *
      </wsdl:output>
      <wsdl:fault name="ncname"> *
        <wsdl:documentation .... /> ?
        <!-- binding details --> *
      </wsdl:fault>
    </wsdl:operation>
  </wsdl:binding>
  <wsdl:service name="ncname" serviceType="qname"> *
    <wsdl:documentation .... /> ?
    <wsdl:port name="ncname" binding="qname"> *
      <wsdl:documentation .... /> ?

    <!-- address details -->
  </wsdl:port>
</wsdl:service>
</wsdl:definitions>

```

UDDI (Universal Description, Discovery and Integration protocol)

UDDI (Universal Description, Discovery, and Integration) เป็นมาตรฐานที่ให้ชุดพื้นฐาน APIs (Application Programming Interface) ของ SOAP ที่สามารถนำมาใช้ในการพัฒนาเป็นตัวแทนของผู้ให้บริการ (Service broker) UDDI ใช้สำหรับค้นหา Service ที่ต้องการและเมื่อได้มาแล้ว UDDI ยังจัดหาข้อตกลงในวิธีการที่จะใช้งานเปรียบได้กับสมุดหน้าเหลือง เป็นมาตรฐานที่จัดตั้งขึ้นโดยบริษัทไอบีเอ็ม บริษัทไมโครซอฟต์ และบริษัทอริบา (Ariba) ปัจจุบันมีบริษัทที่ร่วม กันกำหนดมาตรฐานของ UDDI มากกว่า 70 บริษัท ซึ่งมาตรฐานของ UDDI ถูกกำหนดให้เป็นมาตรฐานสำหรับ B2B interoperability



พนักงานไอทีของ Business 3:

1. ค้นหาข้อกำหนดการให้บริการใน WSDL ผ่านอินเทอร์เน็ตของ Business 1
2. ติดต่อผ่านอินเทอร์เน็ตของแอปพลิเคชันที่ให้บริการ (ด้วย SOAP) ผ่านทางอินเทอร์เน็ต
- 3,4. Business 1 รวมหรือบูรณาการกับ Business 3 ด้วยวิธีเดียวกัน

รูปที่ 2.5 แสดง Web Services with SOAP, UDDI และ WSDL

จากมาตรฐานทั้ง 4 อย่างที่กล่าวข้างต้นสามารถสรุปลำดับขั้นตอนการทำงานของ Web Services ได้ดังนี้

1. Provider จัดทำระบบหรือบริการที่เป็น Web Services ขึ้นมา
2. ทำการลงทะเบียน Web Services กับหน่วยงานที่ให้บริการระบบ UDDI (หรือ Registry)
3. นำ WSDL ไปไว้ในระบบ UDDI ที่ได้ลงทะเบียนไว้
4. Requestor ทำการค้นหาระบบหรือบริการที่ต้องการจากระบบ UDDI

5. เมื่อ Requestor ได้พบระบบหรือบริการที่ต้องการจะนำไฟล์ WSDL ไปเรียนรู้วิธีการเรียก

ใช้ผ่านระบบของตน

6. Requestor ทำการติดต่อและเรียกใช้ระบบหรือบริการจาก Provider ได้โดยตรงผ่าน SOAP

ในระบบของตน

การทำเว็บเซอร์วิส เป็นการแยกส่วนของข้อมูล (Data) และวิธีการทำงาน (Method) ออกจากกัน โดยที่ 1 เว็บเซอร์วิสอาจจะประกอบไปด้วย 1 วิธีการทำงาน หรือมากกว่าก็ได้ขึ้นอยู่กับความต้องการของเรา

ข้อดีอย่างหนึ่งของการแยกข้อมูล และวิธีการทำงานออกจากกันก็คือ จะช่วยให้เราสามารถปรับปรุง แก้ไขวิธีการทำงาน โดยที่เราไม่ต้องไปยุ่งเกี่ยวกับส่วนของข้อมูลแต่อย่างใด

คำว่า วิธีการทำงาน เปรียบเทียบได้ว่า การติดต่อกับฐานข้อมูล ก็ถือเป็น 1 วิธีการ, การแสดงเวลาในฝั่งเซิร์ฟเวอร์ ก็ถือเป็น 1 วิธีการ, การแสดงข้อความต่างๆ ก็ถือเป็น 1 วิธีการ, การบวกเลข ก็ถือเป็นอีก 1 วิธีการ ฯลฯ จะเห็นได้ว่าวิธีการทำงานดังกล่าวขึ้นอยู่กับว่า เว็บเซอร์วิสของเรามีจุดประสงค์ในการทำงานในด้านใด และเมื่อรวมกับการทำงานผ่านเว็บ จึงถูกเรียกว่า เว็บเซอร์วิส

ถ้าพิจารณาลักษณะการทำงานและความสามารถของ Web Services แล้ว เราสามารถสรุปได้ว่า Web Services ก็คือ วิวัฒนาการอีกก้าวหนึ่งของ Web Application นั่นเอง

2.3.4 การประยุกต์ใช้ Web Services กับธุรกิจยุคใหม่

เราสามารถนำ Web Services มาประยุกต์ใช้ในธุรกิจ เพื่อตอบสนองต่อความต้องการบริการแบบ One Stop Service ได้อย่างง่ายดาย เนื่องจาก Web Services มีความสามารถในการแลกเปลี่ยน ข้อมูลและประสานการทำงานบนระบบที่ต่างกันได้ โดยการติดต่อกับผู้ใช้ผ่านทาง Web Application ซึ่งการประยุกต์ใช้ในลักษณะนี้ผู้ใช้ จะรู้เพียงว่า เมื่อเข้ามาที่เว็บไซต์ของเรา ก็จะได้รับบริการที่ครบวงจรเช่นบริการการท่องเที่ยว, บริการสถานที่พักผ่อนและสันทนาการ เป็นต้น

ในเรื่องของความปลอดภัย เนื่องจาก Web Service ทำงานอยู่บน Internet ซึ่งปัจจุบันมีเทคโนโลยีในการรักษาความปลอดภัยมากมายรองรับอยู่แล้ว และ Web Services สามารถผ่านระบบรักษาความปลอดภัย (Firewall) ได้เนื่องจาก SOAP ถูกส่งโดยผ่านโปรโตคอล HTTP นอกจากนี้ Web Services ยังมีระบบรักษาความปลอดภัยตามมาตรฐานของ PKI (Public Key Infrastructure) เช่น MD5 (Message Digest), SSL (Secure Socket Layer) และ PGP (Pretty Good

Privacy) ทำให้มั่นใจได้ว่าการใช้ Web Services เป็นเครื่องมือธุรกิจมีมาตรฐานในการรักษาความปลอดภัยรองรับ

ส่วนเครื่องมือที่ใช้ในการพัฒนา Web Services ก็มีอยู่มากมาย เนื่องจาก Web Services เป็น มาตรฐานที่พัฒนาได้ง่าย เพราะอยู่ในรูปแบบของข้อมูลตัวอักษรหรือ ภาษา XML ทำให้มีชุดเครื่องมือที่สนับสนุนการพัฒนา Web Services มากมาย ซึ่งสามารถแบ่งออกเป็น 4 กลุ่มใหญ่ได้ดังนี้

1. ชุดเครื่องมือที่สนับสนุนโดย Microsoft ตาม Platform ของ Microsoft .NET

Framework

2. ชุดเครื่องมือที่สนับสนุนโดย Sun Microsystems ตาม Platform ของ Sun ONE (Sun

Open

Net Environment)

3. ชุดเครื่องมือที่สนับสนุนโดย IBM เช่น Web Services Toolkit

4. ชุดเครื่องมืออื่นๆที่สนับสนุน SOAP, XML ทั้งที่เป็น Commercial Product และ Open Source

2.4 เปรียบเทียบระหว่าง Web Application และ Web Services

เมื่อเราเข้าใจความหมายและการทำงาน Web Application และ Web Services แล้ว จะเห็นว่าเครื่องมือทั้งสองต่างใช้ HTTP Protocol หรือ อินเทอร์เน็ต เป็นช่องทางในการสื่อสารเหมือนกัน แต่มีวัตถุประสงค์ต่างกัน โดย Web Application ใช้เพื่อการแลกเปลี่ยนไฟล์ HTML ระหว่างเว็บเซิร์ฟเวอร์ แต่ Web Services เป็นการแลกเปลี่ยน “บริการ” (ก้อนโปรแกรม Software Components) ระหว่างระบบสารสนเทศผ่านเว็บเซิร์ฟเวอร์

ในเรื่องของความสามารถโดยส่วนใหญ่จะใช้ Web Application ในการติดต่อกับผู้ใช้ผ่านทาง Internet browser เพื่อนำเสนอข้อมูลและการทำงานต่างๆ ส่วน Web Services จะทำหน้าที่ในการติดต่อกับ เว็บเซิร์ฟเวอร์ เพื่อแลกเปลี่ยนข้อมูลและการทำงานหรือใช้บริการข้ามระบบกันโดยใช้ Web Application หรือ Application Interface ในการติดต่อกับผู้ใช้ นอกจากนี้ Web Services ยังสามารถทำงานกับระบบต่างๆ ได้มากกว่า 1 ระบบ ในขณะที่ Web Application ไม่สามารถทำได้โดยตรง ซึ่งสามารถสรุปการเปรียบเทียบได้ดังตาราง

หัวข้อ	Web Service	Web Application
การเชื่อมต่อ	program – program	human – program
ภาษาที่ใช้	XML	HTML
รายชื่อการให้บริการ	ค้นผ่าน UDDI	ค้นหาผ่าน search engine
ขอบเขตการใช้งาน	Business- to-Business (B2B)	Business-to-Customer (B2C)
โปรโตคอล (Protocol)	SOAP+HTTP	HTTP

ตารางที่ 1 แสดงการเปรียบเทียบระหว่าง Web Services กับ Web Application

หลังจากที่ได้ทำความเข้าใจความหมายและข้อเปรียบเทียบระหว่าง Web Application และ Web Services แล้ว ความแตกต่างระหว่างเครื่องมือทั้งสองจะเห็นได้ชัดในเรื่องของความสามารถ แต่ในการนำไปใช้งานจำเป็นจะต้องประยุกต์ใช้เครื่องมือทั้งสองชนิดร่วมกัน เพื่อให้เกิดประสิทธิภาพมากที่สุด